Escuela Colombiana de Ingeniería Julio Garavito

Desarrollo Orientado por Objetos

Informes de Análisis Dinámico (JaCoCo) y Estático (PMD)

Proyecto SlotMachine — Ciclo 5

Autores:

Juan Espitia

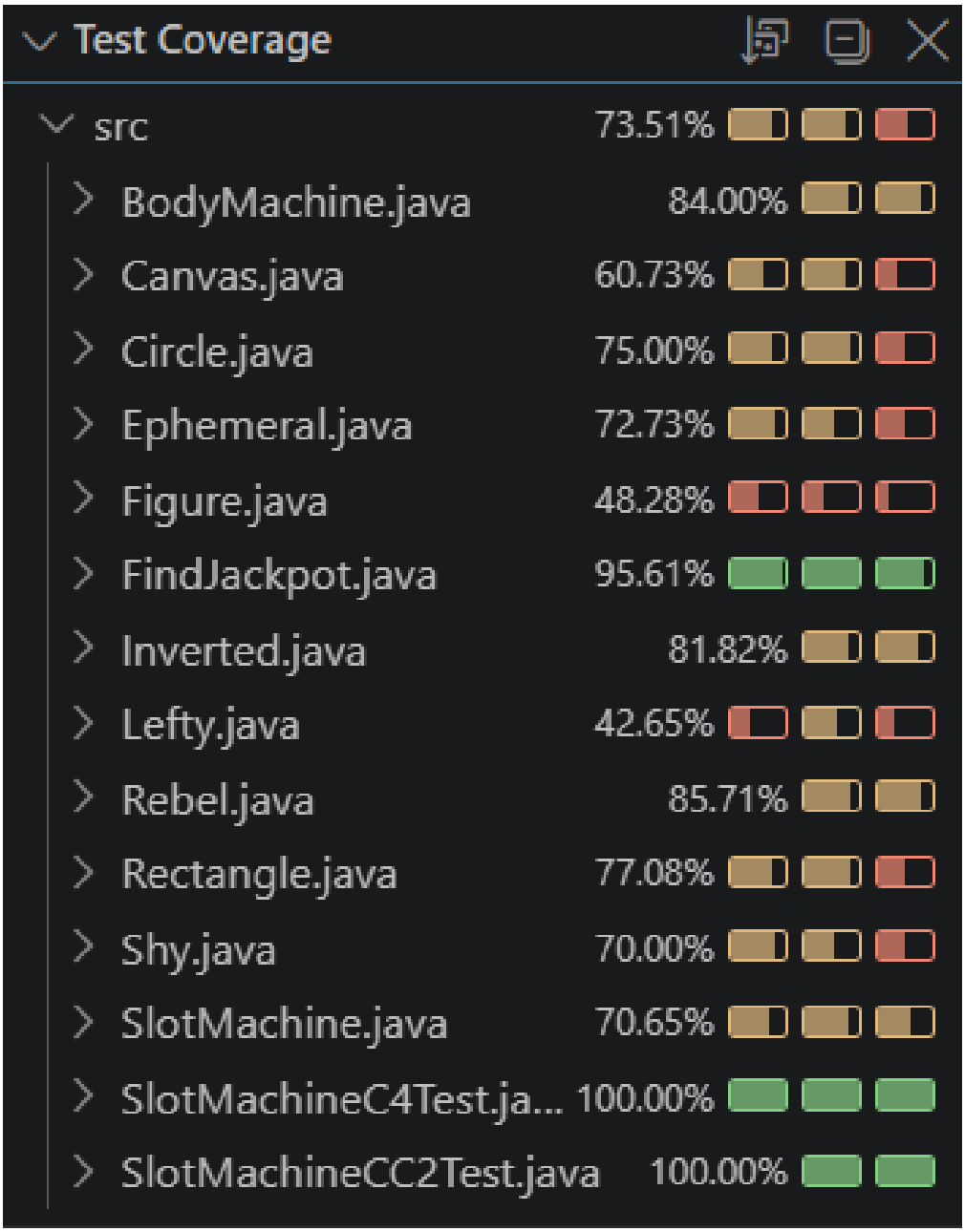
Cristian Salamanca

24 de octubre de 2026

Informes pruebas JaCoCo

Inicialmente es necesario entender a qué se refiere el término JaCoCo, el cual significa Java Code Coverage, el cual es una cobertura para el código a analizar, en este caso todo el proyecto correspondiente a SlotMachine, en donde se hace un análisis dinámico el cual realiza las pruebas unitarias y diagnostica qué tanto están probando el código en su totalidad, por lo cual cuando se corren las pruebas `@Test`, JaCoCo se conecta a la JVM para así realizar su debido diagnóstico. Al terminar JaCoCo determina cuáles sensores de los que utilizan sí pasaron el programa, contra los que quedaron sin tocar.

Para esto, al tener ya el proyecto en el entorno de VS Code se tuvo que instalar una extensión con el nombre Extension Pack for Java, y la que se ejecutó específicamente fue Test Runner for Java, en donde de forma inicial se obtuvieron los siguientes resultados:



Como se puede ver de forma inicial el total del proyecto contaba con un diagnóstico del 73.51 %, en donde las clases con menor porcentaje fueron:

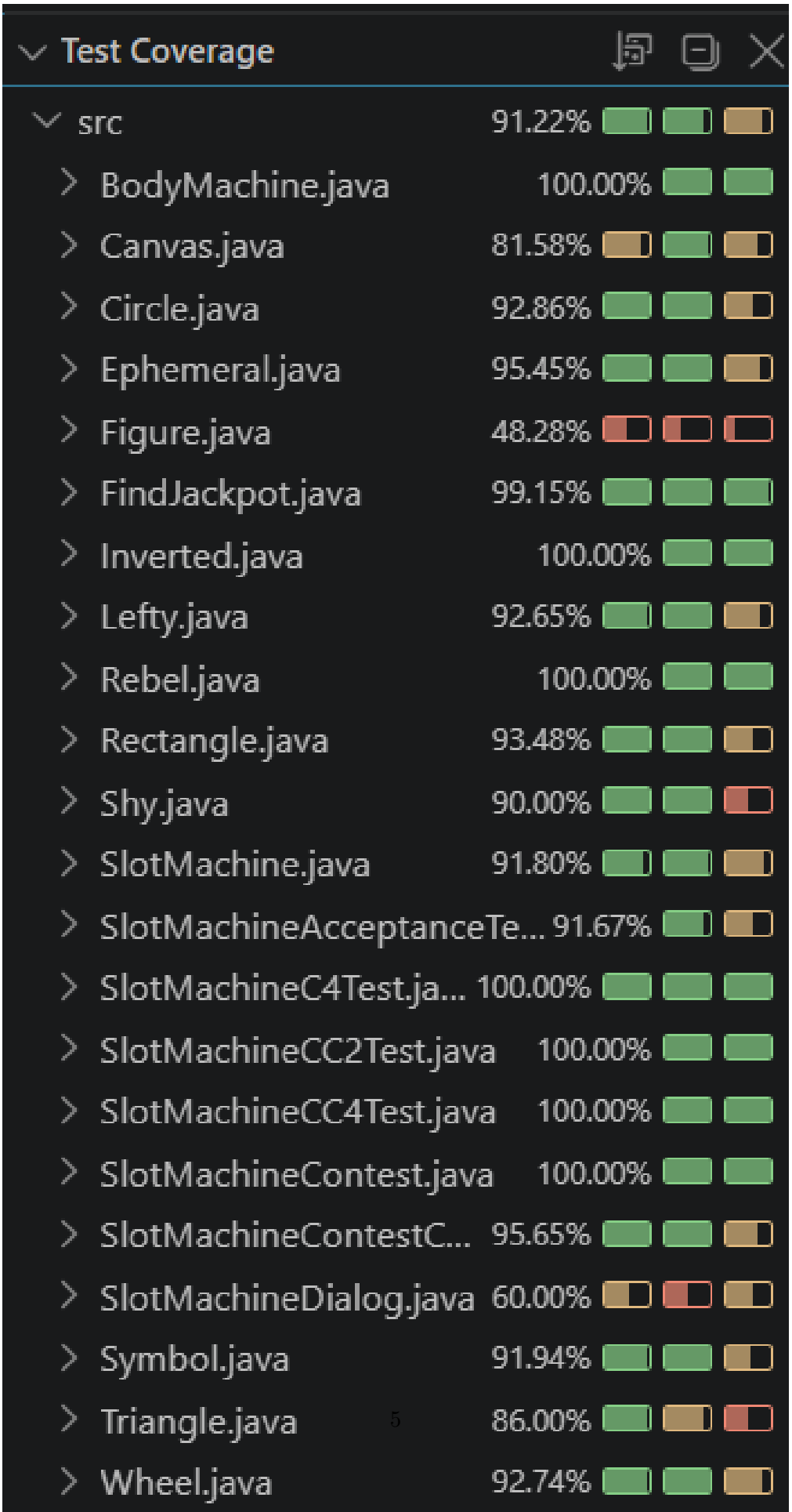
Clase	Porcentaje inicial
`SlotMachineDialog.java`	25.00 %
`Lefty.java`	42.65 %
`Figure.java`	48.28 %
`Canvas.java`	60.73 %
`SlotMachineContest.java`	63.64 %
`Wheel.java`	65.86 %
`Triangle.java`	68.00 %
`Shy.java`	70.00 %
`SlotMachine.java`	70.65 %
`Ephemeral.java`	72.73 %

En estas clases se demostraba una falta de cobertura por parte de las pruebas de unidad, en donde faltaba abarcar métodos presentes en cada clase, por lo cual se añadieron nuevas pruebas con un resultado más favorecedor. Se añadieron siete pruebas nuevas que le dieron más cobertura al código, y los cambios se enfocaron principalmente en:

Prueba añadida	Enfoque
`shouldNotSpinLockedLeftyWheel`	Se encarga de probar que si una rueda Lefty está en estado bloqueado, no copie el símbolo de su vecino izquierdo.
`shouldSpinLeftyInAllModes WithAndWithoutNeighbor`	Se encarga de probar qué ocurre cuando Lefty está en la posición uno sin vecina, prueba un giro aleatorio, luego se crea una segunda copiando a su vecina, activando el decrement del símbolo Ephemeral. Además prueba su constructor parametrizado.
`shouldLockAndUnlockNormal WheelInAllSpinModes`	En las pruebas iniciales nunca se llamaba el método unlock ni se probaba una rueda bloqueada, acá ya se comprueba que se mantenga inmóvil.
`shouldExerciseEphemeralShy AndSymbolOperations`	Lo que hace es probar el ciclo que se hace durante los spins, en donde se hace una visibilidad temporal y luego pasa a un estado invisible, además se verifica el decrement para Ephemeral y que Shy alterne su visibilidad.
`shouldExerciseWheel MethodsDirectly`	Hace pruebas de métodos internos de las clases relacionadas a las Wheels.
`shouldHandleBoundaryAnd EmptyCasesInSlotMachine`	Lo que hace es probar la validación de SlotMachine en modo invisible.
`solveDirectoYCasosLimite`	Invoca directamente `contest.solve(3)` lo cual con anterioridad no era llamado en ningún test.

Adicionalmente, como decisión de diseño se optó por no cubrir en las pruebas unitarias el método `exit()` de `SlotMachine` (ya que ejecuta `System.exit(0)` y detendría la ejecución de JUnit y JaCoCo), ni forzar los diálogos de `SlotMachineDialog` que están diseñados exclusivamente para el modo visible.

Luego de los cambios vistos anteriormente se volvió a realizar la prueba por el JaCoCo, en donde ahora el porcentaje final fue del 91.22 %:


Test Coverage
src 91.22%
BodyMachine.java 100.00%
Canvas.java 81.58%
Circle.java 92.86%
Ephemeral.java 95.45%
Figure.java 48.28%
FindJackpot.java 99.15%
Inverted.java 100.00%
Lefty.java 92.65%
Rebel.java 100.00%
Rectangle.java 93.48%
Shy.java 90.00%
SlotMachine.java 91.80%
SlotMachineAcceptanceTe... 91.67%
SlotMachineC4Test.ja... 100.00%
SlotMachineCC2Test.java 100.00%
SlotMachineCC4Test.java 100.00%
SlotMachineContest.java 100.00%
SlotMachineContestC... 95.65%
SlotMachineDialog.java 60.00%
Symbol.java 91.94%
Triangle.java 86.00%
Wheel.java 92.74%

Para concluir se puede afirmar que la comprobación mediante la herramienta JaCoCo fue de gran soporte para así volver más robusto el apartado de pruebas, además de que en la implementación de las nuevas pruebas, se permite ver de forma más clara, probables errores que con anterioridad no estaban contemplados ya que eran silenciosos y no eran perceptibles sin las pruebas añadidas a través del cumplimiento de este requisito correspondiente al proyecto.

Informe pruebas PMD

Para empezar con el informe acerca del analizador de código PMD hay que definirlo, la cual es una herramienta de análisis estático, lo que indica que el mismo hace una inspección del código fuente, que para el caso específico del proyecto serían los archivos `.java` sin necesidad de ejecutarlos, esto permite encontrar malas prácticas, código muerto e incumplimientos de los estándares de Java. Para su uso en VS Code, fue necesaria la instalación de la extensión PMD for Java, la cual utiliza el ruleset `quickstart.xml` clasificando las reglas por severidad de la siguiente forma:

Severidad	**Descripción**
Prioridad 1 (rojo/error)	Problemas críticos de estándar o diseño.
Prioridad 3 (Amarillo/Warning)	Sugerencias de estilo.

Resultado inicial

Al ejecutar el análisis por primera vez dentro del entorno de trabajo, salieron un total de 117 alertas totales, en donde 13 alertas correspondían a prioridad alta y las demás a prioridad 3. Las alertas de prioridad 1 correspondían a lo siguiente:

Cantidad	**Clase**	**Descripción de la alerta**
10	`SlotMachine`	Por la regla `LocalVariableNamingConventions` en donde las variables locales estaban escritas en `snake_case`, lo cual no está permitido según los estándares de Java, esto sucedía en variables como: `principal_body`, `max_wheel`, `body_x`, `body_width` y `max_symbol`.
2	`Canvas`	Por la regla `ClassWithOnlyPrivate ConstructorsShouldBeFinal` en donde las clases `Canvas` y `CanvasPane` tenían constructores privados pero no eran `final`.
1	`FindJackpot`	Por la regla `MethodNamingConventions` en donde el método `solve_base` tenía guion bajo en vez de ser `camelCase`.

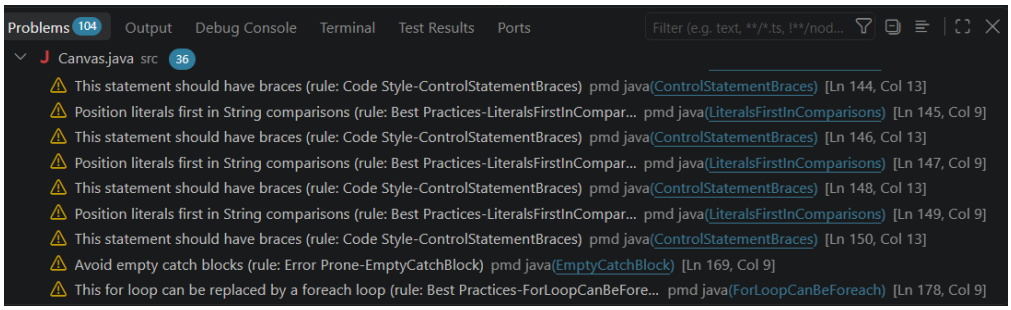
Decisiones tomadas

Se tomó la decisión de corregir principalmente las alertas de prioridad 1 en donde los cambios fueron:

Elemento	Cambio realizado
`SlotMachine`	Pasar todas las variables locales de `SlotMachine` a `camelCase` (`principalBody`, `maxWheel`, `bodyX`, `bodyWidth` y `maxSymbol`).
`FindJackpot`	Renombrar `solve_base` a `solveBase` en `FindJackpot`, `SlotMachineContest`, `SlotMachineContestCC2Test` y en los diagramas de Astah, quitando de paso el modificador `public` redundante en la interfaz.
`Canvas` y `CanvasPane`	Agregar `final` a `Canvas` y `CanvasPane`.

Como limpiezas adicionales se eliminó `import java.util.HashMap` el cual no tenía uso alguno dentro de `SlotMachine`, también el atributo `type` el cual era redundante dentro de `Wheel` y por último los `return` vacíos tanto en `Rebel` como en `Symbol`. Por otro lado, para la prioridad 3 no se hicieron mayores cambios para no afectar la estructura del proyecto proveniente de BlueJ para que a la hora de la ejecución de las pruebas unitarias no generara mayor problema, como con el caso de `NoPackage`. Además para mantener la correspondencia estricta con los métodos de diagrama de diseño de Astah no se agregaron métodos privados adicionales en `SlotMachine`.

Resultado final



Como se evidencia finalmente quedaron en total 0 errores de prioridad 1 y las 38 pruebas unitarias siguen pasando en verde tras la refactorización realizada a través de la extensión PMD for Java, lo cual deja en evidencia que los cambios fueron realizados con éxito y ayudaron a pulir el proyecto en cuanto a los estándares requeridos en el lenguaje de programación Java.